CSC 311 Winter 2024 Final Project Report

Yumna, Ethan, Helia, Jiho

April 7, 2024

Contributions:

All group members independently attempted each question of part A before we came to an agreement on what our solutions should be. Then we split up the workload of writing up solutions to part A as evenly as possible. Everyone also tried to modify each algorithm for part B before coming together to decide on solutions and how to spilt the work. Yumna completed all of question 1 of part A as well as question 1 and 4 of part B. Helia completed part A question 3 part b, c, and d, as well as part B question 3. Jiho wrote code for the autoencoder model of part A and B, and completed question 1 and 2 of part B. Ethan completed question 2 of part A, question 5 of part B, and did the formatting for the final report.

Part A:

1. Collaborative filtering with k-Nearest Neighbor:

In this part, you will implement collaborative filtering using the k-Nearest Neighbor (kNN) algorithm. The starter code is in `part_a/knn.py`. The `knn_impute_by_user` function in `knn.py` implements **user-based collaborative filtering**. Given student A, kNN finds another student B who is most similar to student A in terms of the correctness of their answers and predicts student A's correctness using student B's correctness. The core underlying assumption is that if student A has the same correct and incorrect answers on other diagnostic questions as student B, A's correctness on specific diagnostic questions matches that of student B.

(a) Complete the main function in `knn.py` to run user-based collaborative filtering. Run user-based collaborative filtering for $k \in \{1, 6, 11, 16, 21, 26\}$. Plot the validation accuracy as a function of k. Report the k value you selected and the test accuracy of the final model.

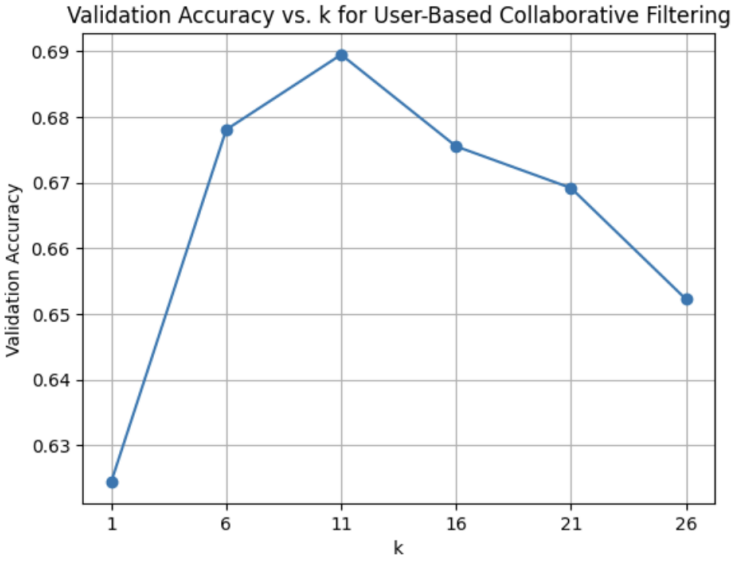


Figure 1: Graph showing the validation accuracy for User-Based Collaborative Filtering

- Best k found: 11
- Final validation accuracy with best k: 0.6895286480383855
- Final test accuracy with best k: 0.6841659610499576

(b) Implement the function `knn_impute_by_item` to perform **item-based collaborative filtering**. Given question A, kNN finds the closest question B that was answered similarly and predicts the correctness of answering question A based on the correctness of answering question B. State the underlying assumption on item-based collaborative filtering.

- The underlying assumption is that similar items have similar patterns of interactions with users. This assumption implies that if a user interacts positively (or negatively) with one item, they are likely to interact positively (or negatively) with other items that are similar to it.

(c) Repeat part (a) with item-based collaborative filtering.

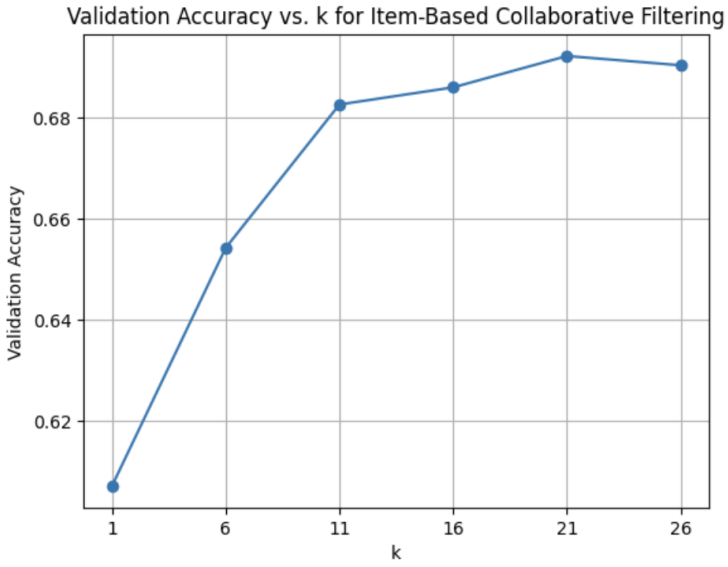


Figure 2: Graph showing the validation accuracy for Item-Based Collaborative Filtering

- Best k found: 21
- Final validation accuracy with best k: 0.6922099915325995
- Final test accuracy with best k: 0.6816257408975445

(d) Which one of user-based and item-based collaborative filtering is better? Why?

- The item-based collaborative filtering shows a higher validation accuracy when $k = 21$ and this indicates that this model fits the training data well and is able to generalize better for the validation dataset. However, user-based collaborative filtering outperforms item-based collaborative filtering due to its higher test accuracy. Additionally, user-based collaborative filtering achieves a higher test accuracy with a smaller k value of 11, making it more efficient and cost-effective for generalizing missing values in the dataset.
- Both models show a lower testing accuracy than the validation accuracy, showing evidence of overfitting. However, the difference between the test and validation accuracy is larger for the item-based collaborative filtering, indicating that this model shows greater tendency to overfit compared to user-based collaborative filtering. Hence, user-based collaborative filtering is a better model to use overall.

(e) List two potential limitations of kNN for the task you are given.

- There may be cases where because we select student B's questions based on student's A's interaction with questions, this may lead to certain questions that would've helped student B in his/her learning but were not chosen because student A did not interact well with this, for example, he may have just ignored the question because he was not paying attention to the assessment. This highlights a limitation of collaborative filtering, where the recommendations are solely based on past interactions without considering the reasons behind them.
- Scattered datapoints is another limitation of kNN for the task given. In cases where the dataset of questions is very dense and the similarities between questions are scattered, the nearest neighbor representing the closest question may not be sufficiently close in terms of relevance. This can lead to suboptimal recommendations and potentially decrease the effectiveness of the collaborative filtering approach.

2. The Item Response Theory Model:

In this part, you will implement an Item-Response Theory (IRT) model. The starter code is in `part_a/item_response.py`. In the one-parameter IRT model, student i has an ability θ_i and question j has the difficulty β_j. Then, the probability that student i answers question j correctly is given by:

$$p(c_{ij} = 1|\theta_i, \beta_j) = \frac{\exp(\theta_i - \beta_j)}{1 + \exp(\theta_i - \beta_j)}$$

(a) Derive the log-likelihood $\log p(\mathbf{C}|\boldsymbol{\theta}, \boldsymbol{\beta})$ for all students and questions, where $\mathbf{C}$ is the sparse matrix. Then, derive the derivative of the log-likelihood with respect to θ_i and β_j (Hint: recall the derivative of the logistic model with respect to the parameters).

- Let N be the number of users and D be the number of questions. Also assume that all logarithms are natural logarithms (base e) for the purposes of making the calculations simpler.
- First we must determine $p(c_{ij}|\theta_i, \beta_j)$. We already know $p(c_{ij} = 1|\theta_i, \beta_j)$, so we need to calculate $p(c_{ij} = 0|\theta_i, \beta_j)$.

$$\begin{aligned} p(c_{ij} = 0|\theta_i, \beta_j) &= 1 - p(c_{ij} = 1|\theta_i, \beta_j) \\ &= 1 - \frac{\exp(\theta_i - \beta_j)}{1 + \exp(\theta_i - \beta_j)} \\ &= \frac{1}{1 + \exp(\theta_i - \beta_j)} \end{aligned}$$

So $p(c_{ij}|\theta_i, \beta_j)$ is:

$$p(c_{ij}|\theta_i, \beta_j) = \left(\frac{\exp(\theta_i - \beta_j)}{1 + \exp(\theta_i - \beta_j)}\right)^{c_{ij}} \left(\frac{1}{1 + \exp(\theta_i - \beta_j)}\right)^{1-c_{ij}}$$

- Using this formula, we can calculate the log likelihood, $\log p(\mathbf{C}|\boldsymbol{\theta}, \boldsymbol{\beta})$:

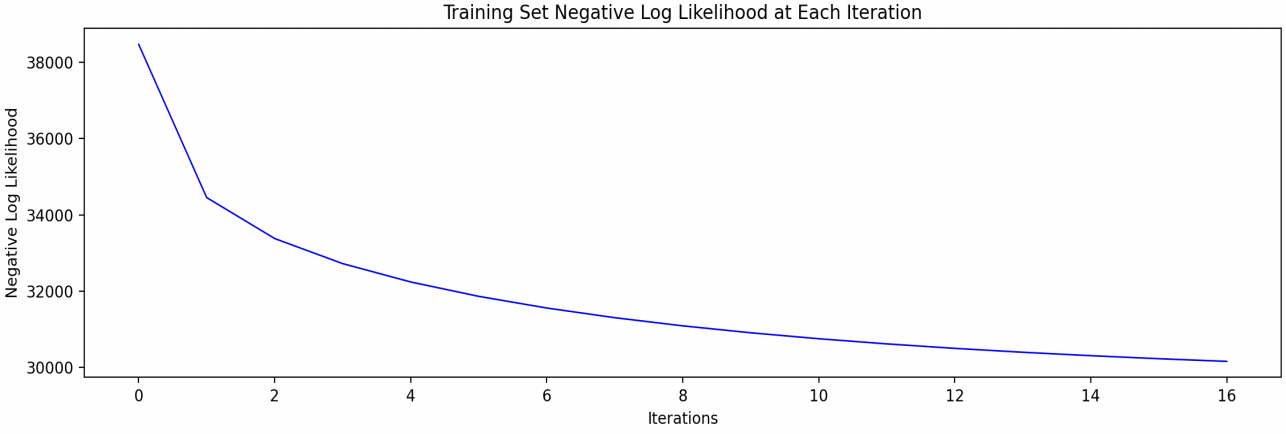
$$\begin{aligned} \mathcal{L}(\mathbf{C}|\boldsymbol{\theta}, \boldsymbol{\beta}) &= \prod_{i=1}^{N} \prod_{j=1}^{D} p(c_{ij}|\theta_i, \beta_j) \\ \log p(\mathbf{C}|\boldsymbol{\theta}, \boldsymbol{\beta}) &= \log \left(\prod_{i=1}^{N} \prod_{j=1}^{D} \left(\frac{\exp(\theta_i - \beta_j)}{1 + \exp(\theta_i - \beta_j)}\right)^{c_{ij}} \left(\frac{1}{1 + \exp(\theta_i - \beta_j)}\right)^{1-c_{ij}} \right) \\ &= \sum_{i=1}^{N} \sum_{j=1}^{D} \left[c_{ij} \log \left(\frac{\exp(\theta_i - \beta_j)}{1 + \exp(\theta_i - \beta_j)}\right) + (1 - c_{ij}) \log \left(\frac{1}{1 + \exp(\theta_i - \beta_j)}\right) \right] \\ &= \sum_{i=1}^{N} \sum_{j=1}^{D} \left[c_{ij} \Big(\theta_i - \beta_j - \log(1 + \exp(\theta_i - \beta_j))\Big) + (1 - c_{ij}) \Big(0 - \log(1 + \exp(\theta_i - \beta_j))\Big) \right] \\ &= \sum_{i=1}^{N} \sum_{j=1}^{D} \left[c_{ij}\theta_i - c_{ij}\beta_j - \log(1 + \exp(\theta_i - \beta_j)) \right] \end{aligned}$$

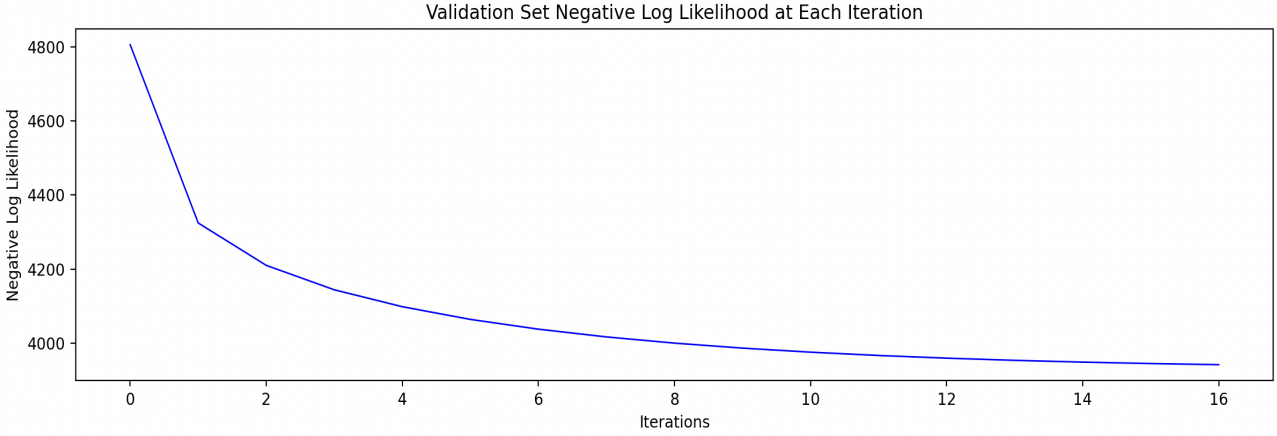
- Now we can take the derivatives of the log-likelihood with respect to the model parameters:

$$\begin{aligned}
\frac{\partial \log p(\mathbf{C}|\boldsymbol{\theta}, \boldsymbol{\beta})}{\partial \theta_i} &= \frac{\partial}{\partial \theta_i} \left(\sum_{i=1}^{N} \sum_{j=1}^{D} \left[c_{ij}\theta_i - c_{ij}\beta_j - \log(1 + \exp(\theta_i - \beta_j)) \right] \right) \\
&= \sum_{j=1}^{D} \frac{\partial}{\partial \theta_i} \left[c_{ij}\theta_i - c_{ij}\beta_j - \log(1 + \exp(\theta_i - \beta_j)) \right] \\
&= \sum_{j=1}^{D} \left[c_{ij} - \frac{\exp(\theta_i - \beta_j)}{1 + \exp(\theta_i - \beta_j)} \right] \\
\frac{\partial \log p(\mathbf{C}|\boldsymbol{\theta}, \boldsymbol{\beta})}{\partial \beta_j} &= \frac{\partial}{\partial \beta_j} \left(\sum_{i=1}^{N} \sum_{j=1}^{D} \left[c_{ij}\theta_i - c_{ij}\beta_j - \log(1 + \exp(\theta_i - \beta_j)) \right] \right) \\
&= \sum_{i=1}^{N} \frac{\partial}{\partial \beta_j} \left[c_{ij}\theta_i - c_{ij}\beta_j - \log(1 + \exp(\theta_i - \beta_j)) \right] \\
&= \sum_{i=1}^{N} \left[-c_{ij} + \frac{\exp(\theta_i - \beta_j)}{1 + \exp(\theta_i - \beta_j)} \right] \\
&= -\sum_{i=1}^{N} \left[c_{ij} - \frac{\exp(\theta_i - \beta_j)}{1 + \exp(\theta_i - \beta_j)} \right]
\end{aligned}$$

(c) Tune the hyperparameters (the learning rate and number of iterations) and report the hyperparameters you selected. With your chosen hyperparameters, plot the training and validation log-likelihoods as a function of iteration. Report the validation and test accuracies for your final model.

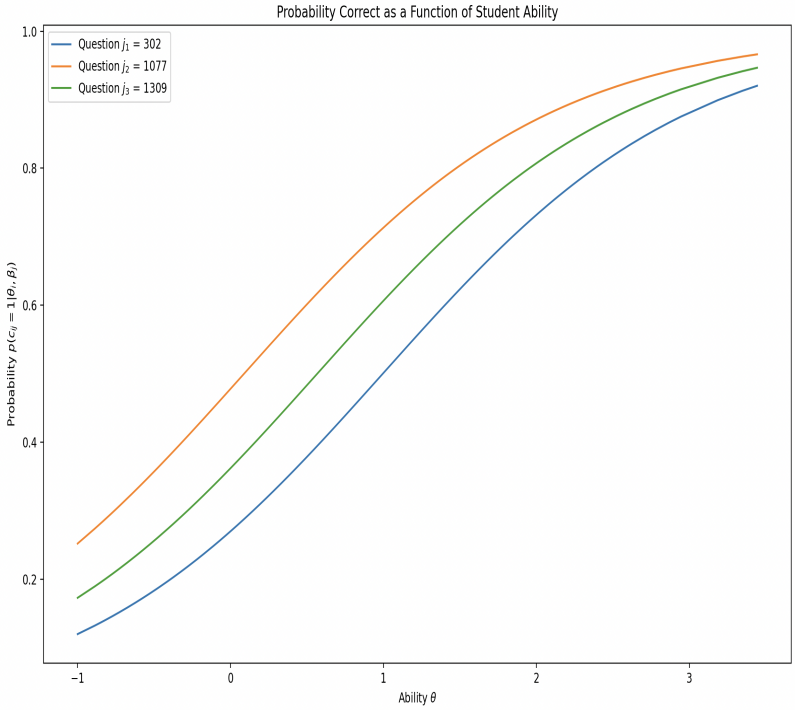
- Selected learning rate: 0.012
- Selected number of iterations: 16
- Final Validation Accuracy: 0.7064634490544736
- Final Test Accuracy: 0.7047699689528648
- We would like to note that the final validation accuracy is not the best value we can get when tuning the hyperparameters, but when we stop at a higher validation accuracy we get a lower final test accuracy which means we are overfitting. So the selected learning rate and number of iterations were chosen to mitigate this tradeoff and get the best validation accuracy while maintaining the best test accuracy. Also note that we only tested after the model was trained so we wouldn't affect the training process.





(d) Select three questions j_1, j_2, and j_3. For each question, plot a curve showing the probability of the correct response $p(c_{ij} = 1)$ as a function of the ability θ using the trained θ and β. Put all three curves on the same figure. Comment on the shapes of the curves and briefly describe what these curves represent.

- The shapes of the three curves resemble transformed sigmoid functions. This is because the probability that student i got question j correct is calculated by taking sigmoid$(\theta_i - \beta_j)$.
- These curves represent the probability that a student with ability θ will get the specific question j_1, j_2, or j_3 correct (depending on which curve we are considering). These curves show us that as a student's ability θ increases, so does the probability that they will get a question correct. Note that the curves are not the same sigmoid function, which is because each question has a different difficulty β which transforms the curves. For example, question $j_1 = 302$ has a higher difficulty β than the other curves since for each student's ability, the probability that the student will get question j_1 correct is always less than the probability of getting j_2 or j_3 correct.
- Note that we choose the questions randomly and so when running the code in `item_response.py`, you will likely get different graphs, but they will all have the similar sigmoid shape. We did this so that it wouldn't look like we were hand selecting which curves to show because of some bias we had.



3. Neural Networks:

In this part, you will design an autoencoder model. Given a user $\mathbf{v} \in \mathbb{R}^{N_{\text{questions}}}$ from a set of users S, our objective is:

$$\min_{\boldsymbol{\theta}} \sum_{\mathbf{v} \in S} \|\mathbf{v} - f(\mathbf{v}; \boldsymbol{\theta})\|_2^2,$$

where f is the reconstruction of the input $\mathbf{v}$. The network computes the following function:

$$f(\mathbf{v}; \boldsymbol{\theta}) = h\left(\mathbf{W}^{(2)} g\left(\mathbf{W}^{(1)}\mathbf{v} + \mathbf{b}^{(1)}\right) + \mathbf{b}^{(2)}\right) \in \mathbb{R}^{N_{\text{questions}}}$$

where g and h are the logistic (sigmoid) activation function. Here, $\mathbf{W}^{(1)} \in \mathbb{R}^{k \times N_{\text{questions}}}$ and $\mathbf{W}^{(2)} \in \mathbb{R}^{N_{\text{questions}} \times k}$, where $k \in \mathbb{N}$ is the latent dimension. The starter code in PyTorch is at `part_a/neural_network.py`.

(b) Train the autoencoder for $k \in \{10, 50, 100, 200, 500\}$, and tune other hyperparameters (the learning rate and number of iterations). Report the hyperparameters that you selected.

- We began by selecting k, the learning rate, and the number of iterations from a wide range of values. We ran the comparator function with different k values, learning rates, and 200 iterations and plotted the trend.
- As expected, a learning rate of 0.1 led to over-fitting and divergence after a few iterations, while a learning rate of 0.001 reached 68% accuracy at around 200 iterations, and a learning rate of 0.01 achieved the same accuracy in a smaller number of iterations.

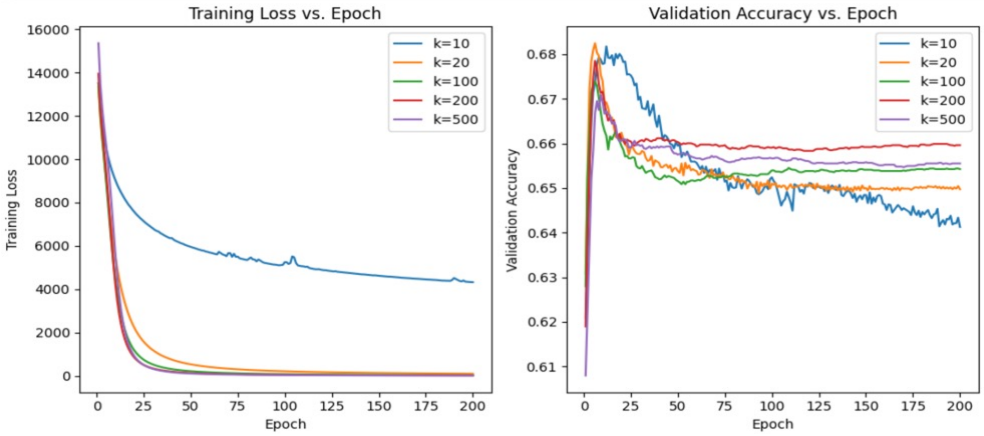


Figure 3: Training loss and validation accuracy using a learning rate of 0.1

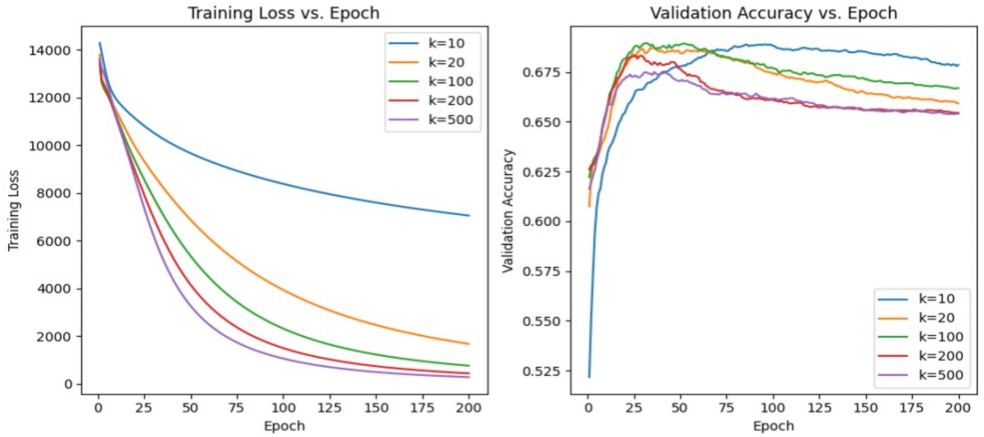


Figure 4: Training loss and validation accuracy using a learning rate of 0.01

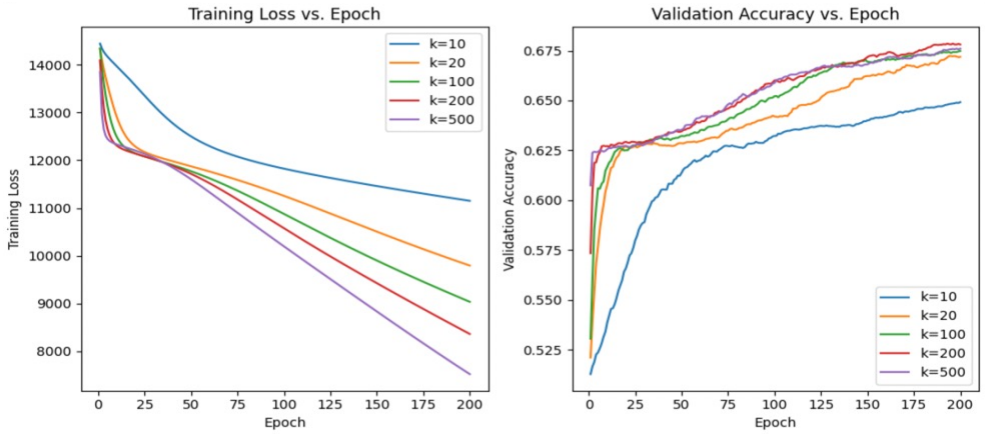
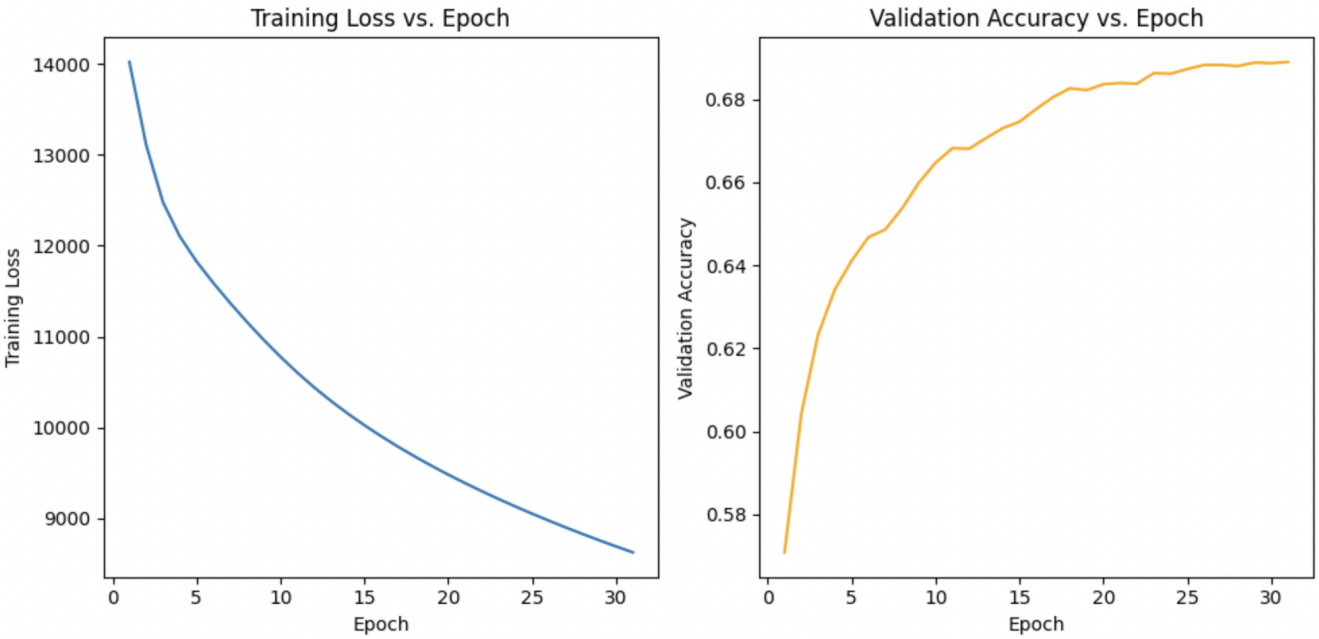


Figure 5: Training loss and validation accuracy using a learning rate of 0.001

- Using the above plots we chose k to be 10 or 100 since these values gave us the best accuracies. Although we still needed to consider the trade-off between more iterations and more hidden layer values. Therefore, we ran the comparator function with more specific values $k \in \{10, 100\}$ and learning rate $\in \{0.1, 0.2, 0.3, 0.4, 0.5\}$.
- Subsequent runs revealed that maintaining a learning rate of 0.03 with 31 iterations and $k = 10$ yielded highest accuracy with the least trade-off for time and computational cost.
- So the final hyperparameters we selected are:
  - Learning rate = 0.03
  - Number of iterations = 31
  - $k = 10$

(c) With your chosen hyperparameters, plot the training and validation objectives as a function of epoch. Also, report the test accuracy for the final model.

- Final validation accuracy: 68.89%
- Final test accuracy: 68.61%



(d) Modify the function `train` so that the objective adds the L_2 regularization. The objective is as follows:

$$\min_{\boldsymbol{\theta}} \sum_{\mathbf{v} \in S} \|\mathbf{v} - f(\mathbf{v}; \boldsymbol{\theta})\|_2^2 + \frac{\lambda}{2} \left(\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_F^2 \right)$$

You may use the method `get_weight_norm` to obtain the regularization term. Using the k and other hyper-parameters selected from part (d), tune the regularization penalty $\lambda \in \{0.001, 0.01, 0.1, 1\}$. With your chosen λ, report the validation and test accuracies of the final model. Does your model perform better with the regularization penalty?

- After tuning the model with our regularizer, we selected the regularization penalty λ to be 0.001. After training the model with the new objective we got the following accuracies:
  - Final validation accuracy: 68.82%
  - Final test accuracy: 68.78%
- We found that the regularizer didn't have a significant impact on our accuracy. The regularized model has a slightly decreased validation accuracy and slightly increased test accuracy. The validation accuracy decreased by 0.07% and the test accuracy increased by 0.17% which are both insignificant in the grand scheme of things.

Part B: Modified Algorithm

1. Formally Describe Your Modified Algorithm

- We decided to modify the neural networks algorithm from part A. With our model from part A, the performance was lacking because we faced issues of overfitting and optimization difficulties.
- To enhance the prediction accuracy, we incorporated question metadata into the neural network. The question metadata file contains information about the subject of each question, specifying the area of mathematics it covers. We utilized this file to create a matrix containing the probabilities of students correctly answering questions related to specific subjects. We chose this as one of our modifications because it allows us to more accurately fit our parameters to the underlying patterns in the data, which would in theory mitigate the effects of overfitting. The process of creating this matrix involved several steps:
    1. Calculating subject-wise responses (`student_subject_matrix` in our code): By referencing the training data matrix, we determined the number of questions a student responded to and the number of correct responses for each subject.
    2. Calculating subject-wise probability (`subject_probabilities` in our code): For each subject, we computed the fraction of correct responses out of the total number of responses, representing the probability of a student answering questions from that subject correctly.
    3. Determining associated subjects: We identified all subjects associated with each question, considering that questions may cover multiple subjects.
    4. Computing average probability (`predicted_correctness` in our code): We averaged the probabilities of correctly answered questions related to all associated subjects for each question, representing the overall likelihood of a student answering questions from those subjects correctly.
- We anticipated that using the "`predicted_correctness`" matrix as the additional input to the autoencoder would enhance its performance. However, after experimenting with various approaches, we discovered that utilizing "`subject_probabilities`" yielded better results.
- In our modified approach, we incorporated both the "`zero_train_matrix`" and the "`subject_probabilities`" matrix as inputs to the autoencoder. Specifically, we fed the rows of these matrices into separate linear layers ("g" and "$g_subject$"), each followed by a sigmoid activation function. The outputs from these layers were then summed together and passed through another linear layer ("h"), followed by a sigmoid function, to produce the final result. **To see the computation graph of our modified model as described above, see figure 7**
- By integrating the "`subject_probabilities`" matrix into the autoencoder model in this manner, we aimed to provide the model with more informative features, ultimately improving its ability to predict students' responses to diagnostic questions more accurately.
- Through extensive research on how to optimize the autoencoder model, we came to a decision to modify the algorithm by also using dropout layers. Dropout is a regularization technique used in neural networks to mitigate the risk of overfitting, a common issue where the model fits too closely to the training data and fails to generalize well to unseen data. It operates by randomly "dropping out" a portion of neurons during training, temporarily removing them along with their connections from the network. **For a visualization of a neural network model with and without dropout layers, see figure 6**.
- Since our neural network from part A is prone to overfitting, regularization methods such as dropout are used during training to reduce the training error and improve the overall performance of the model. Given that we were using L_2 regularization for our neural network in part A, research articles like [1] reported that dropout layers are known to outperform L_2 regularization, hence why we decided to modify the algorithm to gauge whether there was an improvement in the model's overall performance using dropout instead of L_2 regularization.
- Our neural network has 2 hidden layers and so we added two dropout probabilities, one for the input layer and one for the first hidden layer of 0.5 and 0.5 respectively. A dropout probability of 0.5 for the input layer means that during training, each unit in the input layer has a 50% chance of being dropped out (set to zero). Similarly, a dropout probability of 0.5 for the hidden layer means that during training, each unit in the second hidden layer has a 50% chance of being dropped out.

[1] Sitaula, C., & Ghimire, N. (2020). An Analysis of Early Stopping and Dropout Regularization in Deep Learning.

2. Figure or Diagram

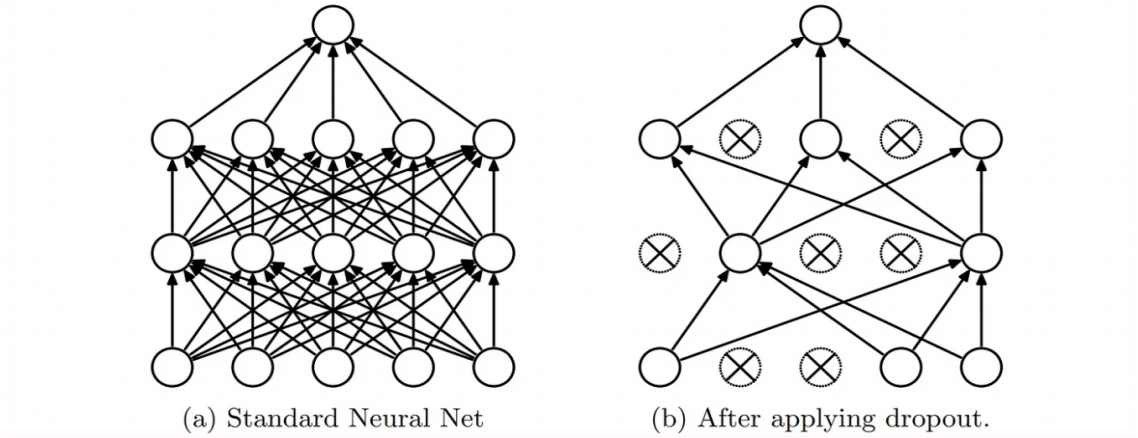


Figure 6: Source: `https://medium.com/analytics-vidhya/neural-network-and-dropouts-b6690c869a18`

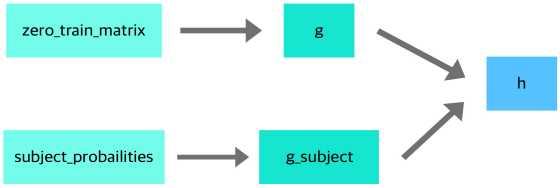


Figure 7: Diagram of our modified autoencoder

3. Your Model versus Baseline Models

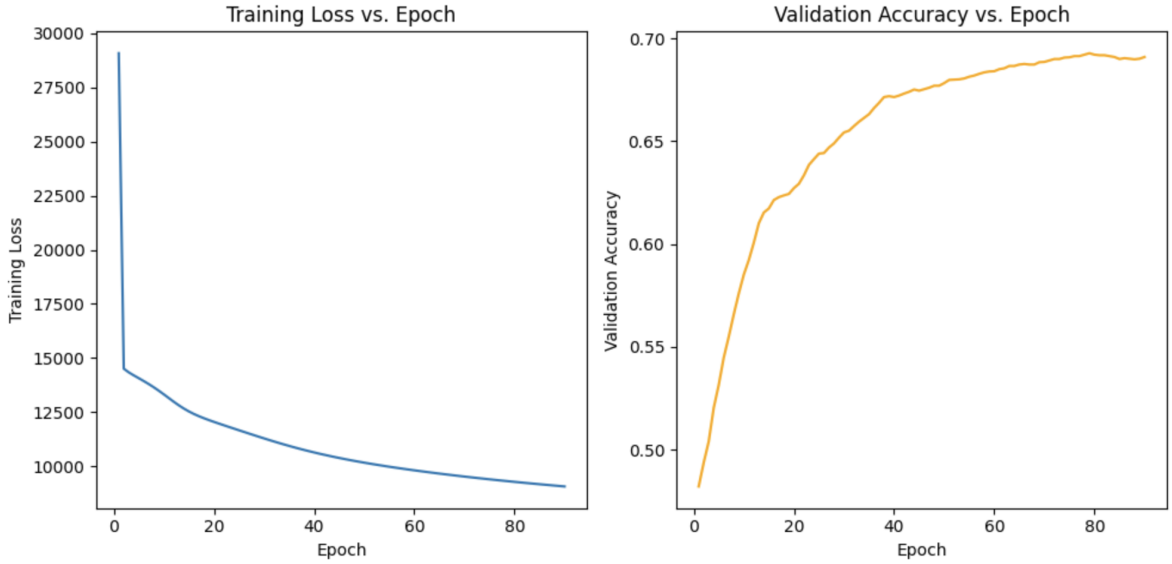


Figure 8: Training Loss and Validation Accuracy for Modified Neural Network

Method	Validation accuracy	Test accuracy	Computation Cost
User-Based kNN	68.95%	68.41%	Low
Item-Based kNN	69.22%	68.16%	Low
Item Response Theory	70.64%	70.47%	Moderate
Neural Network	68.89%	68.61%	Moderate
Neural Network with Regularizer	68.82%	68.78%	Moderate
Modified Neural Network	69.50%	69.63%	High

Table 1: Comparison of Different Methods

Modified Neural Network vs. Baseline Models:

- User-based kNN and item-based kNN:
  - The modified neural network significantly outperforms both user-based kNN and item-based kNN in terms of accuracy on both the validation and test sets.
  - While the kNN models have lower computational costs, they fail to capture complex patterns in the data as effectively as neural networks, resulting in lower accuracies.
- Item Response Theory Model:
  - While Item Response Theory (IRT) shows the highest accuracies among all models, the modified neural network's performance is comparable, with a slightly lower test accuracy but still performing competitively.
  - IRT may require a similar amount of computational resources due to its statistical modelling complexities. However, in our comparisons, it appeared to run faster than the modified neural network.
- Basic Neural Network and Neural Network with Regularizer:
  - The modified neural network outperforms both the neural network with regularizer and without regularizer in terms of accuracy on both the validation and test sets.
  - The modifications made to the neural network architecture lead to improvements in accuracy compared to the baseline models, indicating the effectiveness of the changes. However, it made the model more computationally expensive. Nevertheless, we are willing to make that tradeoff for the improved accuracy.
- The modified neural network is the preferred choice among the compared methods due to its slightly higher accuracy; approximately one percent better than the baseline models.
- While the improvement isn't massive, it's a noteworthy enhancement over the baseline models, demonstrating the effectiveness of the modifications.
- The modified neural network emerges as the top-performing model among the compared methods, with a test accuracy of 69.63%, slightly outperforming the baseline models and showing competitive performance compared to IRT.

4. Explain Your Model's Performance

- Our model performed better than the baseline models since the modified model reached a validation accuracy of 69% at an earlier epoch of 57 as shown in figure 9 below. Our current baseline models reach a validation accuracy of at most 68.82% as reported in part A, question 3. With the use of dropout regularization for both hidden layers, we can infer that the validation accuracy has increased by 1% which indicates that the model is performing better with the added dropout layers and additional "`subject_probabilities`" matrix rather than the previous model from part A.
- In the given context, a dropout probability of 0.10 for both the input and hidden layers implies that approximately 10% of the input units and hidden units will be randomly deactivated during each training iteration. This mechanism prevents the network from becoming overly reliant on any specific input or hidden unit. Consequently, different subsets of neurons are deactivated in each iteration, compelling the network to learn more robust and generalizable features.

```
Epoch: 55       Training Cost: 9922.214396       Valid Acc: 0.6878351679367768
Epoch: 56       Training Cost: 9890.022586       Valid Acc: 0.6891052780129834
Epoch: 57       Training Cost: 9858.607401       Valid Acc: 0.6902342647473892
Epoch: 58       Training Cost: 9827.925809       Valid Acc: 0.6906576347727914
Epoch: 59       Training Cost: 9797.938028       Valid Acc: 0.6907987581145921
Epoch: 60       Training Cost: 9768.606773       Valid Acc: 0.6916454981653966
Epoch: 61       Training Cost: 9739.898099       Valid Acc: 0.6916454981653966
Epoch: 62       Training Cost: 9711.779867       Valid Acc: 0.6910810047981937
Epoch: 63       Training Cost: 9684.222677       Valid Acc: 0.6916454981653966
Epoch: 64       Training Cost: 9657.199343       Valid Acc: 0.6913632514817951
Epoch: 65       Training Cost: 9630.684889       Valid Acc: 0.6907987581145921
Epoch: 66       Training Cost: 9604.654738       Valid Acc: 0.6912221281399944
Epoch: 67       Training Cost: 9579.087181       Valid Acc: 0.691927744848998
Epoch: 68       Training Cost: 9553.961431       Valid Acc: 0.6923511148744003
Epoch: 69       Training Cost: 9529.257869       Valid Acc: 0.6931978549252046
Epoch: 70       Training Cost: 9504.957890       Valid Acc: 0.6941857183178097
Epoch: 71       Training Cost: 9481.044151       Valid Acc: 0.6944679650014113
Epoch: 72       Training Cost: 9457.500216       Valid Acc: 0.6939034716342083
Epoch: 73       Training Cost: 9434.311660       Valid Acc: 0.694609088343212
Epoch: 74       Training Cost: 9411.464435       Valid Acc: 0.6939034716342083
Epoch: 75       Training Cost: 9388.946074       Valid Acc: 0.6941857183178097
Epoch: 76       Training Cost: 9366.745354       Valid Acc: 0.6943268416596104
Epoch: 77       Training Cost: 9344.852286       Valid Acc: 0.6934801016088061
Epoch: 78       Training Cost: 9323.257509       Valid Acc: 0.6926333615580017
Epoch: 79       Training Cost: 9301.951800       Valid Acc: 0.6933389782670054
Epoch: 80       Training Cost: 9280.928147       Valid Acc: 0.6926333615580017
Epoch: 81       Training Cost: 9260.178641       Valid Acc: 0.692492238216201
Epoch: 82       Training Cost: 9239.696235       Valid Acc: 0.6930567315834039
Epoch: 83       Training Cost: 9219.474049       Valid Acc: 0.6944679650014113
Epoch: 84       Training Cost: 9199.506075       Valid Acc: 0.694609088343212
Epoch: 85       Training Cost: 9179.786352       Valid Acc: 0.694609088343212
Epoch: 86       Training Cost: 9160.309370       Valid Acc: 0.6950324583686142
Epoch: 87       Training Cost: 9141.070612       Valid Acc: 0.6943268416596104
Epoch: 88       Training Cost: 9122.065456       Valid Acc: 0.6947502116850127
Epoch: 89       Training Cost: 9103.289659       Valid Acc: 0.6950324583686142
```

Figure 9: The Validation Accuracy reaching 0.69 at epoch 57 and onwards for a dropout probability of 0.10

- By introducing randomness through dropout, the training process discourages the network from memorizing peculiarities in the training data. Instead, it learns the underlying patterns in the data that make the model generalize well to unseen data.
- Therefore, the expedited convergence of the modified model evidenced by reaching a validation accuracy of 69% at epoch 57, compared to the baseline models, suggests that the combined effects of dropout regularization and L_2 regularization led to improved model performance by addressing overfitting and promoting better generalization, ultimately resulting in higher validation accuracy.

5. Analyze One Limitation of Your Model

- One limitation of our modified neural network model is increased training time. Dropout regularization requires the model to remove a portion of neurons and return neurons that were previously removed for each iteration of the training process. For example, if we use dropout probability p on a layer with k neurons, then we expect each iteration to remove approximately pk neurons from the neural network. This creates extra computational complexity with dependence on the number of neurons in our model. This means that our modified neural network with dropout regularization will have a much longer training time than the baseline neural network from part A. It also requires extra training time because for each training example the model requires more forward and backward passes when compared to the baseline model. That is, since we remove neurons for each iteration of the baseline model, we must complete more iterations of the modified model to ensure all of the neurons are properly trained to a similar degree.
- The increased training time also causes a cascade of other limitations like the ability to tune the hyperparameters well. Dropout regularization requires additional hyperparameters (dropout rate and dropout layer placement in the neural network) which means we have to tune those hyperparameters along with the latent dimension k, the learning rate, the number of iterations, and the regularization penalty λ (since we are still using L_2 regularization). Since there are so many hyperparameters for our model, they are very challenging to tune well. On top of this, we must use the training function a lot when tuning hyperparameters and since we use dropout the training time takes much longer and so tuning the hyperparameters well becomes even harder (or at least takes significantly more time than our baseline model).
- One situation where we expect our model to perform poorly is when training using a very large data set that has a considerable amount of noise. This is because dropout performs poorly in regard to time complexity on large data sets. Also since the data set is noisy, we will likely have to use a higher dropout probability to mitigate the risks of overfitting to the training data. This increase in dropout probability will also increase the time complexity of training using dropout regularization.

- This limitation exists because of how dropout regularization is implemented in neural networks. It is also because neural networks are typically modeled using very large data sets which inhibits good training times when using dropout regularization.
- One possible modification we could implement to diminish this limitation is to split the training data set into multiple smaller training data sets and run the model on each subset individually. That is, take the current training data set and split it into l subsets of the training data set and run the model on one of the subsets until it converges to an acceptable accuracy, then take the next subset and run it using the partially trained model parameters until it converges again and repeat this as many times as is necessary. This modification would mitigate the increase in training time complexity since we are using much smaller data sets when training the model and so the dropout regularization has less of an affect on the time complexity.